Rapport de projet 3A : Plateforme Retrofit

Sigma Clermont

presented by

Jean Couronné & Jules Bissinger

Enterprise:
Sigma Clermont
External tutor
Philippe Vaslin
SIGMA Clermont's tutor:
Nicolas Bouton

Date: Février 2026

SIGMA CLERMONT
CAMPUS DE CLERMONT-FERRAND – CS 20265 – 63178 AUBIERE– FRANCE
TEL. +33 (0)4 73 28 80 00 – FAX +33 (0)4 73 28 81 00 –https://www.sigma-clermont.fr/

SUMMARY

SUMMARY 3

PREFACE 4

CHAPTER 1. DESCRIPTION DU SYSTEME 5

CHAPTER 2. RETROFIT 9

2.1. PREMIERES PISTES EXPLOREES 9
2.1.1. Phidget bridge 9
2.1.2. LabJack 9
2.2. SOLUTION RETENUE 10

CHAPTER 3. PREMIERE ACQUISITION 11

3.1. COMPOSANTS ARDUINO 11
3.2. CODE ARDUINO 12

CHAPTER 4. ACQUISITION SIMULTANEE DES SIX VOIES 13

CHAPTER 5. CALIBRATION 15

5.1. PRINCIPE 15
5.2. METHODE EXPERIMENTALE 15

CHAPTER 6. PISTES D'AMELIORATION 17

6.1. FREQUENCE D'ECHANTILLONAGE 17
6.2. FIXATION DES COMPOSANTS 17
6.3. CHARGE MAXIMALE ET REMPLACEMENT DES CAPTEURS 18
6.4. CALIBRATION 18

CHAPTER 7. CONCLUSION 19

CHAPTER 8. ANNEXES 20

8.1. PREMIERE VERSION DU CODE 20
8.2. CODE ARDUINO 23
8.3. CODE PYTHON 24

PREFACE

Ce projet a pour but la remise à niveau technologique d'une plateforme d'acquisition d'efforts. La plateforme composée d'une plaque, d'un support et de 3 capteurs bidirectionnels était capable de récupérer les efforts et moments (par exemple d'une personne se tenant debout dessus), mais le système fonctionnait avec de nombreux composants et sur Windows2000. Le but est alors de simplifier ce système et de pouvoir faire l'acquisition avec un ordinateur actuel (Windows11), voir à terme avec une tablette tactile.

CHAPTER 1. DESCRIPTION DU SYSTEME

Le système est une plaque d'acquisition faite sur mesure, composée d'une plaque en métal (800x600x80 mm) accrochée à un pied « en Y » par l'intermédiaire de 3 capteurs bidirectionnels.

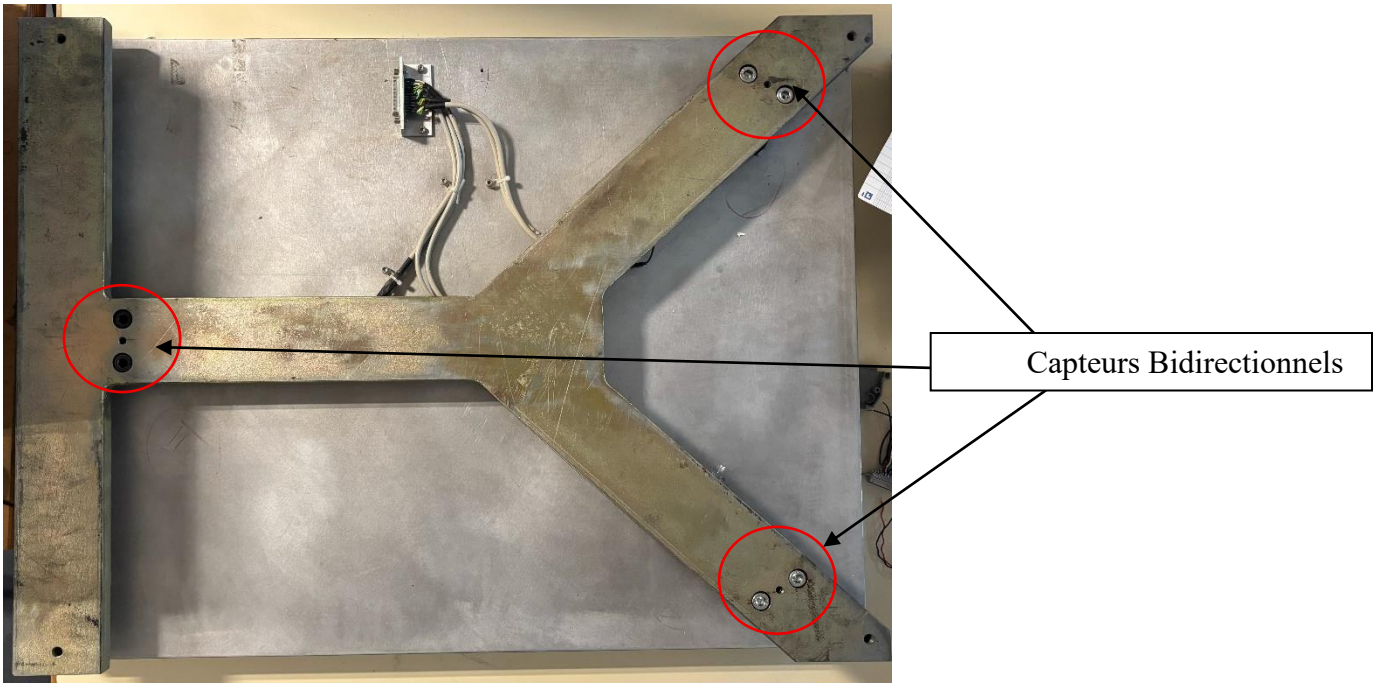

Figure 1 : Plateforme d'acquisition – vue de dessous

Les capteurs sont des capteurs de forces classiques à 4 ports, fonctionnant sur le principe du ***Pont de Wheatstone***.

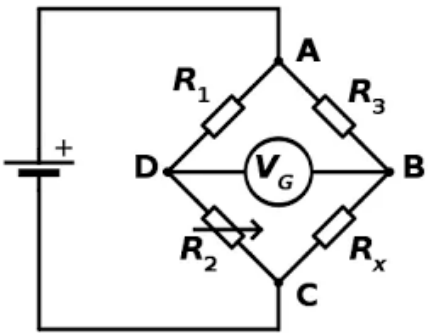

Figure 2 : Pont de Wheatstone

La disposition des capteurs est la suivante : la première direction de tous les capteurs est la verticale, et la seconde est placée de sorte à ''former un cercle'' (Voir ***Deuxième direction des capteurs***).

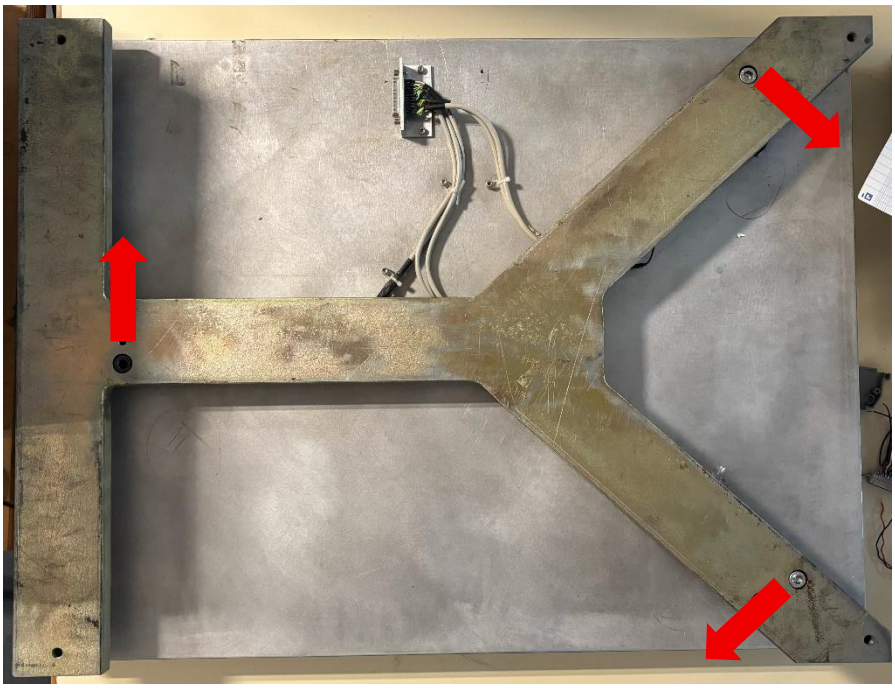

Figure 3 : Deuxième direction des capteurs

Chaque capteur mesure simultanément une composante verticale et une composante horizontale, ce qui conduit à six voies de mesure indépendantes. L'objectif du système est d'obtenir, à partir de ces six signaux, le torseur complet des efforts appliqués à la plaque, soit les trois forces et les trois moments au centre de la structure.

En effet, en plaçant les capteurs de cette manière, on peut avoir accès à tous les efforts et moments avec seulement 3 capteurs (en connaissant l'angle des deux capteurs en biais).

Le fonctionnement global peut être décomposé en quatre étapes successives. Les capteurs constituent la première étape. Ils sont basés sur le principe du pont de Wheatstone et délivrent une variation de tension proportionnelle à la déformation mécanique. Cette variation étant de très faible amplitude, elle nécessite une amplification et une conversion analogique numérique. La seconde étape consiste donc à transformer ces signaux analogiques en valeurs numériques exploitables. Une fois ces six valeurs numériques obtenues, une matrice de transformation permet de calculer les composantes du torseur d'efforts. Enfin, les résultats sont visualisés et enregistrés sur un ordinateur afin d'être exploités pour l'analyse expérimentale.

1.1. Ancien système

Toute la chaîne d'acquisition reposait sur une architecture aujourd'hui obsolète. Les signaux issus des capteurs étaient envoyés vers un boîtier d'amplification volumineux. Le câblage était centralisé via une prise DB25 regroupant les six capteurs. Une carte électronique sur mesure intégrée dans une autre prise DB25 permettait de transformer directement les six signaux bruts en forces et moments.

L’étalonnage s’effectuait physiquement au moyen de potentiomètres ajustés manuellement. Cette méthode présentait plusieurs limites importantes. D’une part, elle rendait la calibration peu reproductible, car dépendante de réglages mécaniques sensibles. D’autre part, toute modification nécessitait une intervention matérielle.

Le traitement et la visualisation étaient assurés par un ordinateur fonctionnant sous Windows 2000, avec une interface développée sous Scilab. Cette dépendance à un environnement informatique ancien rendait l’utilisation du système fragile et difficilement maintenable. L’ensemble formait une architecture volumineuse, peu modulaire et difficilement adaptable aux standards actuels.

CHAPTER 2. RETROFIT

L'objectif du retrofit était de concevoir une solution compacte, pérenne, économiquement raisonnable et simple d'utilisation. Il était indispensable de numériser le traitement des signaux dès l'acquisition afin de supprimer les réglages analogiques et de rendre la calibration reproductible et documentée. Le nouveau système devait être compatible avec un ordinateur moderne et pouvoir évoluer facilement dans le temps.

2.1. Premières pistes explorées

2.1.1. Phidget bridge

Une première expérimentation a été réalisée avec un module Phidget Bridge. Cette solution présentait plusieurs avantages immédiats. Elle intègre directement l'amplification des ponts de Wheatstone et fournit une conversion analogique numérique stable via USB. L'installation et la mise en œuvre ont été rapides, et les premiers essais ont confirmé la faisabilité d'une acquisition numérique fiable des signaux des capteurs.

Cependant, cette solution s'est rapidement révélée insuffisante pour le besoin spécifique de la plateforme. Le nombre de voies disponibles sur un module unique ne permettait pas de traiter simultanément les six directions de mesure. Il aurait été nécessaire d'utiliser deux interfaces distinctes, impliquant deux connexions USB indépendantes vers l'ordinateur. Une telle configuration aurait augmenté la complexité matérielle et logicielle, réduit la compacité recherchée et introduit un risque de désynchronisation entre les deux ensembles de mesures. Bien que techniquement fonctionnelle, cette solution ne répondait pas pleinement aux objectifs du projet.

2.1.2. LabJack

Une autre option étudiée a été l'utilisation d'une carte d'acquisition de type LabJack. Ce type de matériel professionnel offre des performances élevées, une bonne précision et une grande flexibilité en termes d'entrées analogiques. Une telle carte aurait permis de centraliser l'acquisition des six signaux sur un unique périphérique USB, avec une qualité métrologique reconnue.

Cependant, plusieurs éléments ont conduit à écarter cette solution. Le coût d'une carte LabJack adaptée aux besoins du projet s'élève à environ 800 euros. Cette dépense apparaissait disproportionnée par rapport au budget disponible et à la nature académique du projet. Par ailleurs, bien que robuste, cette solution reste relativement fermée et moins flexible qu'une architecture entièrement programmable et open source.

2.2. Solution retenue

Après analyse des différentes options, la solution retenue repose sur une carte Arduino Mega 2560 associée à un Shield Grove et à six modules d'amplification et de conversion HX711. Cette architecture présente plusieurs avantages.

Tout d'abord, elle permet de disposer de six voies indépendantes avec une seule connexion USB vers l'ordinateur. Chaque module HX711 assure l'amplification ainsi qu'une conversion analogique numérique sur 24 bits, offrant une résolution adaptée aux faibles variations de tension générées par les capteurs.

Ensuite, le coût global de la solution reste inférieur à 100 euros, ce qui la rend particulièrement attractive au regard des performances obtenues. La modularité de l'Arduino Mega 2560 permet également d'envisager des extensions futures sans modification majeure de l'architecture.

Enfin, l'intégration avec un traitement logiciel développé en Python offre une grande stabilité face aux évolutions des systèmes d'exploitation, une compatibilité naturelle avec la communication série USB et un accès à des bibliothèques scientifiques performantes pour le traitement matriciel et la visualisation. Cette approche garantit la pérennité du système et facilite son évolution vers des fonctionnalités plus avancées, telles que l'implémentation complète de la matrice de transformation 6×6 ou l'automatisation des procédures de calibration.

Le système modernisé retenu se compose donc des éléments suivants. Les capteurs mécaniques sont conservés sans modification. Les ponts de Wheatstone sont connectés à des modules HX711 assurant l'amplification et la conversion analogique numérique. Ces modules sont pilotés par une Arduino Mega 2560 équipée d'un Shield Grove. L'ensemble communique via USB avec un ordinateur exécutant un programme Python chargé d'effectuer la calibration, l'application de la matrice de transformation, la visualisation en temps réel et l'enregistrement des données.

Cette nouvelle architecture remplace une chaîne analogique complexe et peu maintenable par un système numérique compact, évolutif et économique.

CHAPTER 3. PREMIERE ACQUISITION

3.1. Composants Arduino

Nous allons maintenant détailler le fonctionnement et les composants de tout le dispositif d'acquisition ARDUINO. Nous avons tout d'abord une Arduino Mega 2560.

Sur cette carte a été installé un ***Shield GROVE*** permettant de faire le lien entre les capteurs et la carte.

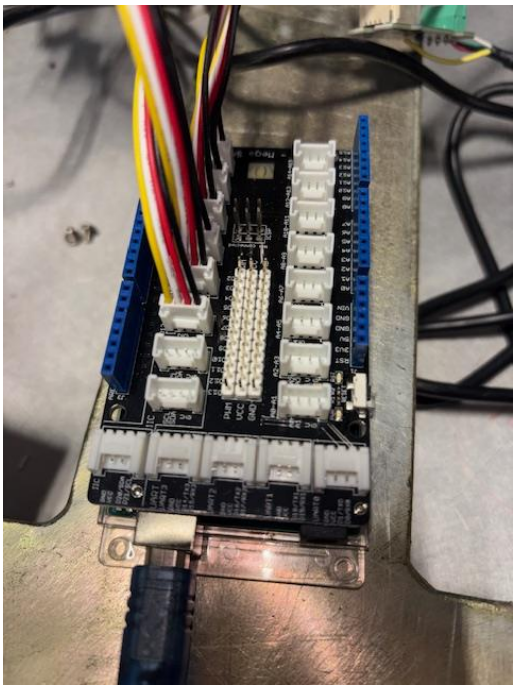

Figure 4 : Shield GROVE

Sur ce Shield sont branchés des modules « GROVE - HX711 », qui sont en réalité des modules récupérant le signal brut du capteur et renvoyant à la carte les valeurs brutes de déformation du capteur [Figure 5].

3.2. Code Arduino

Le code Arduino [Annexe 7.1] est celui de la première acquisition. L'idée ici était de voir s'il était possible de recevoir le signal brut d'un seul capteur avec une variation de la valeur en fonction de la force appliquée. Puis en passant par une calibration à poids connus, avoir une balance capable d'indiquer la masse posée sur le capteur. Ce code a aussi servi à lever l'incertitude concernant les fils sortant des capteurs, dans la mesure où comme nous n'avions pas la documentation desdits capteurs, nous ne savions pas comment les brancher. Grâce à ce code et à un multimètre, nous avons donc réussi à déterminer quels fils correspondaient à quoi.

Figure 5 : Branchement amplificateur Grove

Remarque : dans l'ordre nous avons :
Blanc : Signal –
Jaune : Signal +
Vert : Excitation –
Marron : Excitation +

CHAPTER 4. ACQUISITION SIMULTANEE DES SIX VOIES

La première version du programme avait permis de valider le principe de lecture d'un seul capteur et d'obtenir une conversion simple masse / signal brut après calibration. La seconde permet désormais l'acquisition simultanée des six voies issues des trois capteurs bidirectionnels, rendant possible la reconstruction complète du torseur d'efforts.

Sur la carte Arduino Mega 2560, six modules HX711 sont utilisés, un pour chaque direction de mesure [Figure 3]. Chaque module dispose de deux lignes numériques distinctes, une pour la transmission des données et une pour l'horloge.

La lecture repose sur une implémentation manuelle du protocole du HX711 [Annexe 7.2]. Pour chaque capteur, le microcontrôleur attend que la ligne DOUT passe à l'état bas, indiquant que la conversion est disponible. Vingt-quatre impulsions d'horloge sont ensuite générées afin de récupérer successivement les bits de mesure. Un vingt-cinquième pulse fixe le gain à 128, configuration adaptée au pont de Wheatstone utilisé. La valeur binaire obtenue est enfin convertie en entier. Cette approche bas niveau garantit une maîtrise complète du protocole, sans dépendance à une bibliothèque externe.

Les six valeurs brutes sont envoyées sur le port série sous la forme d'une ligne composée de nombres séparés par des virgules. Ce choix de format simplifie le traitement côté ordinateur tout en assurant une transmission compacte et rapide. Le délai d'acquisition très court permet un échantillonnage quasi continu, limité essentiellement par le temps de conversion interne des HX711 de 80 Hz.

La seconde partie du système repose sur un script Python exécuté sur l'ordinateur [Annexe 7.3]. Celui-ci ouvre la liaison série à 9600 bauds, attend la stabilisation de la carte puis lit en continu les lignes transmises. Chaque ligne est décodée, séparée en six valeurs numériques.

Contrairement à la première version, cette nouvelle implémentation intègre une gestion de la calibration pour chacune des six voies. Au démarrage, le programme recherche un fichier de paramètres contenant les offsets et facteurs d'échelle. Si ces paramètres n'existent pas ou sont incomplets, une procédure interactive de calibration est proposée. Pour chaque capteur, une phase de tarage à vide permet de déterminer l'offset électronique par calcul de la médiane de plusieurs échantillons successifs. Une masse connue est ensuite appliquée et la différence de signal et permet de calculer le facteur d'échelle exprimé en unités par gramme grâce à la formule suivante :

$$Facteur\ d'échelle = \frac{valeur\ brute\ poids\ mesuré - offset}{poids\ connu\ mesuré}$$

L'utilisation d'une médiane sur 20 échantillons plutôt que de la moyenne réduit l'influence d'éventuelles valeurs aberrantes.

Les paramètres obtenus peuvent être sauvegardés dans un fichier au format JSON afin de garantir la reproductibilité des mesures et d'éviter une recalibration systématique. Chaque capteur possède donc son propre offset et son propre coefficient, ce qui permet de compenser les différences de sensibilité des voies.

La conversion en masse est réalisée en temps réel selon la relation reliant valeur brute, offset et facteur d'échelle donnée ci-dessous.

$$Masse = \frac{valeur\ brute\ poids\ mesuré - offset}{Facteur\ d'échelle}$$

Les données converties sont stockées dans des buffers indépendants pour chaque capteur. L'utilisation d'un thread dédié à la lecture série permet de dissocier l'acquisition du traitement graphique, garantissant ainsi la continuité des mesures même lors du rafraîchissement de l'affichage.

L'affichage est assuré par la bibliothèque Matplotlib, avec un tracé simultané des six courbes. Les axes sont ajustés dynamiquement en fonction des valeurs observées et le temps d'exécution est affiché en continu. Cette visualisation permet d'observer directement les sollicitations sur la plateforme.

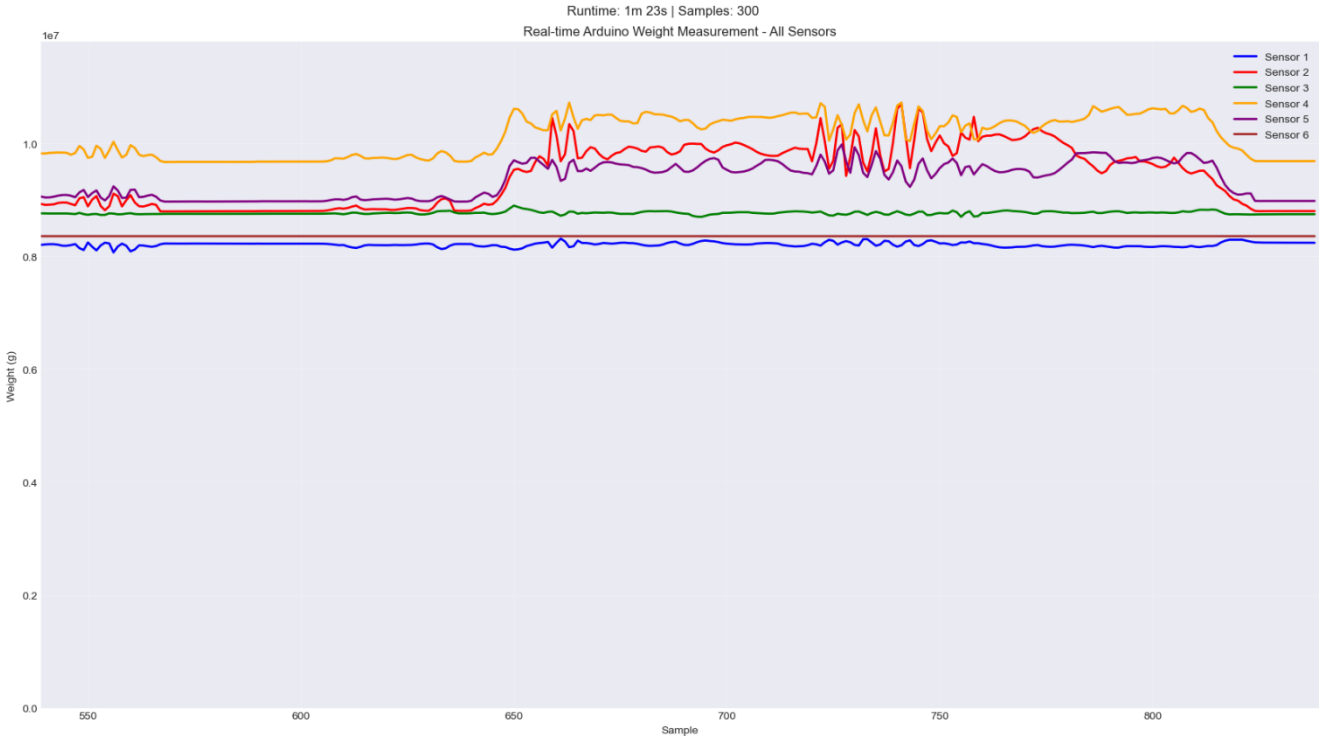

Figure 6 : Acquisition des 6 voies

Un contrôle simple de validité est également appliqué aux données afin d'éliminer des valeurs manifestement incohérentes.

Cette seconde version du code contient ainsi la base logicielle nécessaire à l'implémentation ultérieure de la matrice de transformation permettant de convertir les six mesures indépendantes en un torseur d'efforts complet exprimé au centre de la plateforme.

Le code complet du projet peut être retrouvé sur Github :
https://github.com/Jcouronne/Plateforme_de_mesure

CHAPTER 5. CALIBRATION

5.1. Principe

La configuration géométrique de la plateforme, reposant sur trois points d'appui disposés en triangle équilatéral, permet de définir un système de mesure à six degrés de liberté en exploitant la bidirectionnalité de chaque capteur.

Chaque point de contact mesure simultanément une composante de force verticale et une composante horizontale orientée tangentiellement au cercle circonscrit de la structure. Cette disposition spécifique crée un couplage mécanique entre les six signaux bruts issus des modules HX711 et le torseur des efforts appliqués au centre de la plaque. Afin de traduire ces mesures numériques en grandeurs physiques exploitables, il est nécessaire d'établir une matrice de changement de base, dite matrice de calibration, qui lie les efforts et moments (Fx, Fy, Fz, Mx, My, Mz) aux variations de tension enregistrées. Cette étape de calibration repose sur l'application de charges étalonnées sur les différents axes de la plateforme pour identifier expérimentalement les coefficients de sensibilité et corriger d'éventuels phénomènes de diaphonie entre les voies, assurant ainsi la précision et la fiabilité du nouveau système d'acquisition basé sur l'architecture Arduino.

5.2. Méthode expérimentale

Remarque : la procédure suivante est purement théorique, elle n'a pas été réalisée réellement mais elle a été réfléchie dans l'idée d'une poursuite du projet.

Pour établir la relation entre les signaux numériques et les efforts réels, la méthode expérimentale la plus rigoureuse consiste à identifier les coefficients de la matrice de sensibilité par une série de sollicitations connues. Cette procédure débute par une phase de tarage à vide, permettant de soustraire le poids propre de la plaque métallique et d'annuler les offsets électroniques des modules HX711.
Une fois le zéro initial fixé, on applique successivement des masses étalonnées en des points précis de la plateforme afin de solliciter indépendamment chaque composante du torseur. Par exemple, une masse placée au centre géométrique génère une force Fz pure, tandis que cette même masse déportée d'une distance connue sur l'axe longitudinal induit un moment de flexion My parfaitement calculable. Pour les efforts horizontaux Fx et Fy, on utilise généralement un système de poulies et de masses suspendues permettant d'appliquer une tension parfaitement horizontale sur les tranches de la plaque.

L'accumulation de ces mesures permet de construire un système d'équations linéaires où chaque effort appliqué est mis en regard de la réponse des six voies d'acquisition. Au lieu de se limiter à six mesures isolées, il est préférable de réaliser une montée en charge progressive pour chaque axe afin de vérifier la linéarité des capteurs et d'appliquer une régression par la méthode des moindres carrés.

Le résultat final est une matrice de transfert 6x6 inversée qui, une fois implémentée dans l'algorithme de traitement de l'Arduino, permet de convertir en temps réel les données brutes des capteurs en un torseur d'efforts complet et découplé.

CHAPTER 6. PISTES D'AMELIORATION

Bien que le système modernisé réponde aux objectifs initiaux de compacité et de pérennité logicielle, plusieurs axes d'amélioration peuvent être envisagés afin d'augmenter ses performances, sa robustesse et son potentiel d'exploitation scientifique.

6.1. Fréquence d'échantillonage

Une première limite concerne la fréquence d'échantillonnage imposée par les modules d'acquisition HX711. Ces convertisseurs analogique–numérique offrent deux modes de fonctionnement, dont le plus rapide atteint 80 Hz. Cette valeur constitue actuellement la fréquence maximale d'acquisition du système. Si cette cadence est suffisante pour des mesures statiques ou quasi-statiques, elle peut devenir limitante pour l'étude de phénomènes dynamiques rapides, tels que des impacts, des sauts ou des changements brusques d'appui. Dans ces situations, une fréquence d'échantillonnage plus élevée permettrait de capturer plus fidèlement les variations temporelles des efforts. Une amélioration future pourrait consister à remplacer les modules HX711 par des convertisseurs analogiques numérique plus rapides ou par une carte d'acquisition dédiée offrant un compromis acceptable entre fréquence et coût. Augmenter la fréquence d'échantillonnage permettrait d'élargir le champ d'application expérimental de la plateforme.

6.2. Fixation des composants

Un second axe d'amélioration concerne l'intégration de l'électronique sous la plateforme. Un modèle de plaque support a été commencé sur 3D Experience. Cette plaque imprimée en 3D serait positionnée sous la structure principale afin de maintenir l'ensemble de l'électronique : carte Arduino, modules HX711 et câblage. L'objectif est de maintenir et de sécuriser les composants tout en réduisant les contraintes mécaniques exercées sur les connexions. Ce modèle est actuellement en cours de réalisation et devra être finalisé. [Figure 7]

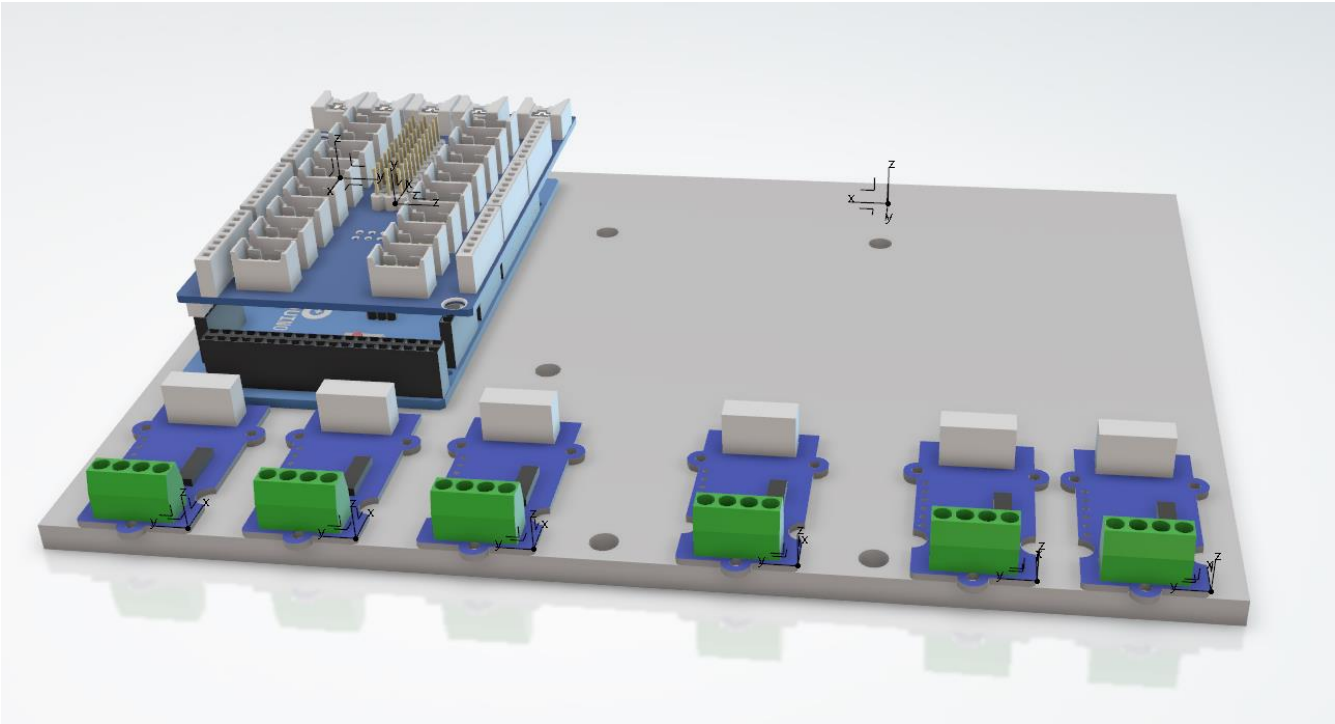

Figure 7 : Plaque support

6.3. Charge maximale et remplacement des capteurs

La plage de mesure des capteurs constitue également une limite importante. Les capteurs actuellement utilisés présentent une capacité maximale d'environ 150 kg. Cette valeur est adaptée à des essais statiques ou à des charges modérées. Toutefois, lors de sollicitations dynamiques les forces instantanées peuvent dépasser largement la capacité maximale des jauges, en particulier lors d'impacts. Une réflexion sur le dimensionnement futur des capteurs pourrait être menée.

Il serait donc pertinent de procéder à un remplacement des capteurs afin de garantir la qualité globale du système.

6.4. Calibration

Enfin, la calibration complète du système représente un axe d'amélioration essentiel. Bien que la méthodologie de calibration ait été présentée précédemment, sa mise en œuvre reste à finaliser. Une procédure rigoureuse, accompagnée d'une analyse d'incertitude, permettrait de quantifier la précision du système.

CHAPTER 7. CONCLUSION

Ce projet de retrofit a permis de transformer une plateforme d'acquisition d'efforts reposant sur une architecture analogique ancienne en un système numérique moderne, compact et évolutif. En conservant l'architecture mécanique et les capteurs d'origine, nous nous sommes concentrés sur la chaîne d'acquisition et de traitement des signaux, avec pour objectif d'améliorer la pérennité, la reproductibilité et la facilité d'utilisation du dispositif.

L'abandon du boîtier analogique et des réglages par potentiomètres au profit d'une acquisition numérique via des modules basés sur le HX711, pilotés par une Arduino Mega 2560, a permis de simplifier considérablement l'architecture matérielle. La transmission unique par USB vers un ordinateur moderne élimine la dépendance à un environnement informatique obsolète et rend le système compatible avec des standards actuels.

Le choix de Python pour le traitement des données permet une gestion claire de la calibration, une application numérique de la matrice de transformation et une visualisation en temps réel des six voies de mesure. Cette approche logicielle améliore la traçabilité des paramètres, facilite les évolutions futures et ouvre la voie à des développements plus avancés, notamment l'automatisation complète de la calibration matricielle ou l'ajout d'une tablette pour utiliser le système.

Le système ainsi obtenu répond aux objectifs fixés en termes de compacité, de coût maîtrisé et de simplicité d'exploitation. Néanmoins, certaines limites subsistent, notamment la fréquence d'échantillonnage limitée à 80 Hz, la capacité maximale des capteurs et la nécessité de finaliser une calibration complète et rigoureuse.

CHAPTER 8. ANNEXES

8.1. Première Version du code

```
1       // Configuration des broches
2       const int DOUT_PIN = 9;
3       const int SCK_PIN = 8;
4
5
6       // Variables globales pour la calibration
7       long offset = 0;           // La valeur à vide (Tare)
8       float facteurEchelle = 1; // Le facteur de conversion pour obtenir des kg
9
10
11      void setup() {
12        Serial.begin(9600);
13        pinMode(SCK_PIN, OUTPUT);
14        pinMode(DOUT_PIN, INPUT);
15        digitalWrite(SCK_PIN, LOW);
16
17        Serial.println("--- CONFIGURATION DE LA BALANCE ---");
18        delay(1000);
19
20
21        // --- ÉTAPE 1 : TARE (Mise à zéro) ---
22        Serial.println("1. TARE : Assurez-vous que le plateau est vide.");
23        Serial.println("   (Calibration en cours dans 3 secondes...)");
24        delay(3000);
25
26        // On prend 10 mesures pour faire une moyenne précise
27        long somme = 0;
28        for(int i=0; i<10; i++) {
29          somme += lireValeurBrute();
30          delay(50);
31        }
32        offset = somme / 10;
33        Serial.print("   -> Tare effectuée. Offset enregistré : ");
34        Serial.println(offset);
35        Serial.println("----------------------------------------");
36
37
```

```
38      // --- ÉTAPE 2 : CALIBRATION (Poids connu) ---
39      Serial.println("2. CALIBRATION :");
40      Serial.println("   Veuillez placer un poids connu sur la balance.");
41      Serial.println("   Ensuite, tapez le poids en KG dans le moniteur série (ex: 0.5 
42
43      // Attente de la saisie utilisateur
44      while (Serial.available() == 0) {
45        // On attend...
46      }
47
48      float poidsConnu = Serial.parseFloat(); // Lire le poids saisi
49
50       if(poidsConnu != 0) {
51         Serial.print("   Calibration avec un poids de : ");
52         Serial.print(poidsConnu);
53         Serial.println(" kg. Mesure en cours...");
54
55         // Mesure du poids connu (Moyenne sur 10 lectures)
56         somme = 0;
57         for(int i=0; i<10; i++) {
58           somme += lireValeurBrute();
59           delay(50);
60         }
61         long lectureAvecPoids = somme / 10;
62
63         // Formule : Facteur = (ValeurLue - Tare) / PoidsRéel
64         facteurEchelle = (float)(lectureAvecPoids - offset) / poidsConnu;
65
66         Serial.print("   -> Calibration terminée. Facteur calculé : ");
67         Serial.println(facteurEchelle);
68       } else {
69         Serial.println("   Erreur : Poids invalide. Le facteur par défaut (1) sera uti
70       }
71
72       Serial.println("----------------------------------------");
73       Serial.println("--- BALANCE PRÊTE ---");
74       delay(1000);
75      }
76
77      void loop() {
78       // --- ÉTAPE 3 : FONCTIONNEMENT NORMAL ---
79
80       // 1. Lire la valeur brute actuelle
81       long lectureActuelle = lireValeurBrute();
82
83       // 2. Calculer le poids réel
84       // Formule : Poids = (Lecture - Tare) / Facteur
```

```
85        float poidsEnKg = (float)(lectureActuelle - offset) / facteurEchelle;
86
87        // 3. Affichage
88        Serial.print("Poids : ");
89        Serial.print(poidsEnKg, 3); // 3 chiffres après la virgule
90        Serial.println(" kg");
91
92        delay(500); // Pause de lecture
93      }
94
95
96      // --- FONCTION DE LECTURE MANUELLE ---
97      long lireValeurBrute() {
98        // Attendre que le capteur soit prêt
99        while (digitalRead(DOUT_PIN) == HIGH) {
100         // On attend que DOUT passe à LOW
101         // (On pourrait ajouter un timeout ici pour éviter un blocage infini)
102       }
103
104
105       unsigned long count = 0;
106
107       // Lecture des 24 bits
108       for (int i = 0; i < 24; i++) {
109         digitalWrite(SCK_PIN, HIGH);
110         delayMicroseconds(1);
111         count = count << 1;
112         digitalWrite(SCK_PIN, LOW);
113         delayMicroseconds(1);
114
115         if (digitalRead(DOUT_PIN)) {
116           count++;
117         }
118       }
119
120       // 25ème pulse (Gain 128)
121       digitalWrite(SCK_PIN, HIGH);
122       delayMicroseconds(1);
123       digitalWrite(SCK_PIN, LOW);
124       delayMicroseconds(1);
125
126   // Conversion (Complément à deux via XOR 0x800000)
127   count = count ^ 0x800000;
128
129   return (long)count;
130 }
131
```

8.2. Code Arduino

```
1  // Configuration des broches
2  const int DOUT_PIN = 3;
3  const int SCK_PIN = 2;
4
5  void setup() {
6    Serial.begin(9600);
7    pinMode(SCK_PIN, OUTPUT);
8    pinMode(DOUT_PIN, INPUT);
9    digitalWrite(SCK_PIN, LOW);
10   delay(1000);
11 }
12
13 void loop() {
14   // Output only the raw sensor value for calibration/processing on PC
15   long lectureActuelle = lireValeurBrute();
16   Serial.println(lectureActuelle);
17   delay(100); // Faster update for smoother PC-side calibration
18 }
19
20 // --- FONCTION DE LECTURE MANUELLE (Votre code encapsulé) ---
21 long lireValeurBrute() {
22   // Attendre que le capteur soit prêt
23   while (digitalRead(DOUT_PIN) == HIGH) {
24     // On attend que DOUT passe à LOW
25     // (On pourrait ajouter un timeout ici pour éviter un blocage infini)
26   }
27
28   unsigned long count = 0;
29
30   // Lecture des 24 bits
31   for (int i = 0; i < 24; i++) {
32     digitalWrite(SCK_PIN, HIGH);
33     delayMicroseconds(1);
34     count = count << 1;
35
36     digitalWrite(SCK_PIN, LOW);
37     delayMicroseconds(1);
38
39     if (digitalRead(DOUT_PIN)) {
40       count++;
41     }
```

```
  }

  // 25ème pulse (Gain 128)
  digitalWrite(SCK_PIN, HIGH);
  delayMicroseconds(1);
  digitalWrite(SCK_PIN, LOW);
  delayMicroseconds(1);

  // Conversion (Complément à deux via XOR 0x800000)
  count = count ^ 0x800000;

  return (long)count;
}
```

8.3. Code python

```
import serial
import threading
import time
import re
import collections
import matplotlib
import matplotlib.pyplot as plt
import sys
import statistics as st
import os
import json

# Configuration
DEFAULT_PORT = 'COM5'  # e.g. 'COM3' or '/dev/ttyACM0'; leave None to be prompted
BAUDRATE = 9600       # Must match Serial.begin(...) in Arduino
BUFFER_SIZE = 300     # number of points to keep in the plot
CALIBRATION_FILE = 'calibration.json'
NUM_SENSORS= 6

# Load or initialize calibration parameters
def load_calibration():
    """Load calibration parameters from file if it exists."""
    if os.path.exists(CALIBRATION_FILE):
        try:
            with open(CALIBRATION_FILE, 'r') as f:
                data = json.load(f)
                offsets = data.get('offset', [None] * 6)
                scales = data.get('facteurEchelle', [None] * 6)
                known_wt = data.get('known_weight', None)
                # Ensure we have arrays for all 6 sensors
                if not isinstance(offsets, list):
                    offsets = [offsets] * 6
```

```
                if not isinstance(scales, list):
                    scales = [scales] * 6
                return offsets[:6], scales[:6], known_wt
        except Exception as e:
            print(f"Error loading calibration file: {e}")
    return [None] * 6, [None] * 6, None

# Initialize calibration parameters
offsets, facteurEchelles, known_weight = load_calibration()
if all(o is not None for o in offsets):
    print(f"Loaded calibration for all 6 sensors")
    for i, (off, scale) in enumerate(zip(offsets, facteurEchelles)):
        print(f"  Sensor {i + 1}: Offset={off}, Scale Factor={scale}")
else:
    print("No complete calibration file found. A new calibration will be created.")

# Serial port selection logic
if DEFAULT_PORT is not None :
    port = DEFAULT_PORT
else:
    port = input('Enter serial port (e.g. COM3 or /dev/ttyACM0): ').strip()

if not port:
    print('No serial port provided. Exiting.')
    sys.exit(1)

# Open serial port
try:
    ser = serial.Serial(port, BAUDRATE, timeout=1)
    time.sleep(2)  # allow Arduino to reset
except Exception as e:
    print(f'Failed to open serial port {port}: {e}')
    sys.exit(1)

print('If you have not already, please complete the calibration steps on the Arduino
(follow serial monitor instructions) before running this script.')
print('Waiting for numeric data from Arduino...')

# Calibration process
calibration = 'n'  # Default: don't perform calibration
if any(o is None for o in offsets):
    calibration = input('Do you want to perform calibration ? (y/n): ')
if calibration == 'y' or calibration == 'Y':
    # Calibration parameters
    CALIBRATION_SAMPLES = 10

    def get_median_raw(sensor_index=0, samples=CALIBRATION_SAMPLES):
        """Get median raw value for a specific sensor."""
        vals = []
        print(f"Collecting {samples} raw samples from sensor {sensor_index + 1}...")
        for i in range(20):
            test=ser.readline()
```

```
            print(test)
        while len(vals) < samples:
            raw = ser.readline()
            try:
                line = raw.decode('utf-8', errors='ignore').strip()
                values = line.split(',')
                if len(values) > sensor_index:
                    val = int(values[sensor_index].strip())
                    print(f"Sensor {sensor_index + 1}: {val}")
                    vals.append(val)
            except:
                continue
        med=st.median(vals)
        print(f"Median raw value (Sensor {sensor_index + 1}): {med}")
        return med

    print("\n--- CALIBRATION ---")
    print("You will calibrate each sensor individually.")
    print("Make sure all sensors have the same load when taring and when placing known weight.")

    for sensor_idx in range(NUM_SENSORS):
        print(f"\n--- Calibrating Sensor {sensor_idx + 1} ---")
        input(f"1. Remove all weight from sensor {sensor_idx + 1} and press Enter to tare...")
        ser.reset_input_buffer()
        offsets[sensor_idx] = get_median_raw(sensor_idx)
        print(f"Tare complete. Offset = {offsets[sensor_idx]}")

        input(f"2. Place a known weight on sensor {sensor_idx + 1} and press Enter...")
        known_weight_sensor = None
        while known_weight_sensor is None:
            try:
                known_weight_sensor = float(input(f"Enter the known weight in g (for sensor {sensor_idx + 1}): "))
                if known_weight_sensor <= 0:
                    known_weight_sensor = None
                    print("Please enter a positive value in grams.")
            except:
                print("Invalid input. Try again.")

        ser.reset_input_buffer()
        raw_with_weight = get_median_raw(sensor_idx)
        print(f"Raw value with weight: {raw_with_weight}")

        if raw_with_weight == offsets[sensor_idx]:
            print(f"Error: No difference between tare and loaded value for sensor {sensor_idx + 1}. Check your hardware.")
            facteurEchelles[sensor_idx] = 1  # Default fallback
        else:
            facteurEchelles[sensor_idx] = (raw_with_weight - offsets[sensor_idx]) / known_weight_sensor
```

```
            print(f"Calibration complete. Scale factor (Sensor {sensor_idx + 1}) = {facteurEchelles[sensor_idx]} (raw units per gram)")
            if abs(facteurEchelles[sensor_idx]) < 0.01:
                print(f"Warning: Scale factor for sensor {sensor_idx + 1} is very small. Calibration may be incorrect.")

    print("\n--- END CALIBRATION ---\n")
    save_choice = input("Do you want to save calibration parameters ? (y/n): ").strip().lower()
    if save_choice == 'y':
        try:
            with open(CALIBRATION_FILE, 'w') as f:
                json.dump({
                    'offset': offsets,
                    'facteurEchelle': facteurEchelles,
                    'known_weight': known_weight
                }, f, indent=4)
            print(f"Calibration parameters saved to {CALIBRATION_FILE}")
        except Exception as e:
            print(f"Error saving calibration file: {e}")

# Shared state
NUM_SENSORS = 6
data = [collections.deque(maxlen=BUFFER_SIZE) for _ in range(NUM_SENSORS)]  # stores floats for each sensor
index = collections.deque(maxlen=BUFFER_SIZE) # x axis (shared)
_lock = threading.Lock()
_running = True
first_data_received = threading.Event()


def reader_thread():
    i = 0
    global _running
    while _running:
        try:
            raw = ser.readline()
            if not raw:
                continue
            line = raw.decode('utf-8', errors='ignore').strip()
            if not line:
                continue
            try:
                # Parse comma-separated values for all 6 sensors
                values = line.split(',')
                if len(values) != NUM_SENSORS:
                    continue

                raw_values = [int(v.strip()) for v in values]

                with _lock:
                    all_valid = True
```

```
                    weights = []

                    # Convert raw values to weights for all sensors
                    for sensor_idx in range(NUM_SENSORS):
                        if offsets[sensor_idx] is not None and
facteurEchelles[sensor_idx] is not None:
                            poids_g = (raw_values[sensor_idx] - offsets[sensor_idx]) /
facteurEchelles[sensor_idx]
                            if poids_g > -50:
                                weights.append(poids_g)
                            else:
                                all_valid = False
                                break
                        else:
                            weights.append(raw_values[sensor_idx])  # Store raw value
if not calibrated

                    # If all sensors are valid, add to data
                    if all_valid:
                        for sensor_idx in range(NUM_SENSORS):
                            data[sensor_idx].append(weights[sensor_idx])
                        index.append(i)

                        print(f"Sample {i}: {', '.join([f'S{j+1}: {w:.1f}g' for j, w in
enumerate(weights)])}")
                        first_data_received.set()
                i += 1
            except Exception as e:
                print(f"Parse error: {e}")
                continue
        except Exception:
            continue


# Start reader thread
t = threading.Thread(target=reader_thread, daemon=True)
t.start()

# Wait for first valid data (i.e., after calibration)
first_data_received.wait()
print('Numeric data received. Plotting started. Press Ctrl+C to stop.')

# Setup matplotlib figure
plt.style.use('seaborn-darkgrid')
fig, ax = plt.subplots(figsize=(12, 6))

plt.ion()  # Turn on interactive mode

# Record start time for runtime display
start_time = time.time()

try:
```

```
    colors = ['blue', 'red', 'green', 'orange', 'purple', 'brown']
    while _running:
        # Clear previous plot
        ax.clear()

        with _lock:
            if len(index) == 0:
                time.sleep(0.01)
                continue
            x = list(index)


        # Plot weight from all sensors
        for sensor_idx in range(NUM_SENSORS):
            with _lock:
                y = list(data[sensor_idx])
            if len(y) > 0:
                ax.plot(x, y, color=colors[sensor_idx], linewidth=2, label=f'Sensor
{sensor_idx + 1}')

        # Set labels and title
        ax.set_title('Real-time Arduino Weight Measurement - All Sensors')
        ax.set_xlabel('Sample')
        ax.set_ylabel('Weight (g)')

        # Update axes limits dynamically
        if len(x) > 0:
            ax.set_xlim(max(0, x[0]), x[-1] + 1)

            # Find min and max across all sensors
            all_values = []
            for sensor_idx in range(NUM_SENSORS):
                with _lock:
                    all_values.extend(list(data[sensor_idx]))

            if all_values:
                ymin = min(0, min(all_values))
                ymax = max(all_values)
                if ymin == ymax:
                    ymax += 10
                ax.set_ylim(ymin, ymax + 0.1 * abs(ymax or 1))

        # Add grid and legend
        ax.legend()
        ax.grid(True, alpha=0.3)

        # Set a common title with running time
        current_time = time.time()
        elapsed_time = current_time - start_time
        elapsed_minutes = int(elapsed_time // 60)
        elapsed_seconds = int(elapsed_time % 60)
        fig.suptitle(f"Runtime:{elapsed_minutes}m{elapsed_seconds}s |Samples:{len(x)}")
```

```
293
294         # Update display
295         plt.tight_layout()
296         plt.pause(1)  # Update display every 1s
297
298 except KeyboardInterrupt:
299     print("Stopping plot...")
300
301 # Shutdown
302 _running = False
303 try:
304     t.join(timeout=1)
305 except:
306     pass
307 try:
308     ser.close()
309 except:
310     pass
311 print('Stopped.')
```